

EIGEN: Efficient and Generalizable Vector Database Tuning with Mini-DBs and LLM-Guided Optimization

Yiyan Li^{*}, Guanli Liu[†], Renata Borovica-Gajic[‡], Andreas Kipf[‡], Cuiping Li^{*}, Hong Chen^{*}

^{*} Renmin University of China, [†] The University of Melbourne, [‡] University of Technology Nuremberg
liyiyang, licuiping, chong@ruc.edu.cn; guanli.liu1, renata.borovica@unimelb.edu.au; andreas.kipf@utn.de

ABSTRACT

Vector databases support modern AI applications such as semantic search, recommendation systems, and retrieval-augmented generation. Their performance is highly sensitive to configuration choices, yet configurations that achieve high query throughput while satisfying recall requirements can vary across database engines, workloads, and hardware environments. Identifying such configurations typically requires repeated and costly evaluations on the full database. We present EIGEN, an efficient and generalizable framework for vector database tuning. To reduce configuration-evaluation cost, EIGEN constructs multiple Mini-DBs via locality-aware stratified sampling and tunes them in parallel. Cross-Mini-DB screening ranks candidates by both performance and stability before validating a small shortlist on the full database. To guide the search, EIGEN introduces CALM, a constraint-aware LLM-guided optimizer that combines knob semantics with measured feedback. Only configurations that satisfy the recall constraint in full-database evaluation are accepted. Engine profiles and execution adapters support reuse across database engines, workloads, and hardware without task-specific pre-training. Across six datasets on Milvus, EIGEN reduces end-to-end tuning time by 98.87% on average compared with GridSearch, the strongest evaluated baseline in terms of observed QPS, and additional experiments on Qdrant and pgvector demonstrate effective transfer across database engines.

PVLDB Reference Format:

Yiyan Li^{*}, Guanli Liu[†], Renata Borovica-Gajic[‡], Andreas Kipf[‡], Cuiping Li^{*}, Hong Chen^{*}. EIGEN: Efficient and Generalizable Vector Database Tuning with Mini-DBs and LLM-Guided Optimization
. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/intlyy/EIGEN>.

1 INTRODUCTION

Vector databases underpin modern AI applications such as semantic search [12], recommendation systems [32], and retrieval-augmented generation (RAG) [16]. Achieving high query throughput while maintaining retrieval quality requires careful configuration across multiple layers of the system [37]. These configuration

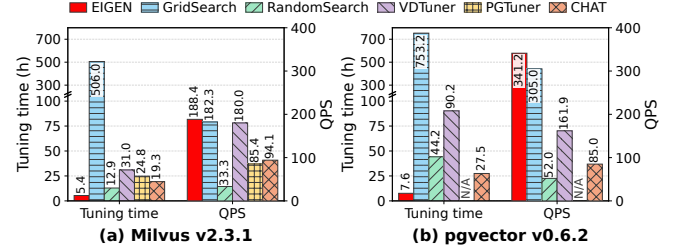


Figure 1: Comparison of end-to-end tuning time and QPS on Tiny5M under recall constraint of 0.975. N/A means a method failed to transfer to the corresponding database engine.

choices broadly span three categories: system knobs, index configurations, and search parameters. *System knobs* govern resource allocation and execution behavior, such as memory usage and query parallelism [34]. *Index configurations* determine how the index is constructed, such as graph connectivity, partition granularity, and compression settings [12, 20]. *Search parameters* control query-time exploration, such as the search width in HNSW, directly affecting the trade-off between QPS and recall [14, 20]. Given a database engine, workload, and hardware environment, vector database tuning seeks an end-to-end configuration across these layers that maximizes QPS while satisfying a target recall constraint [37].

Manually solving this tuning problem is challenging even for experienced database administrators (DBAs). The configuration space is mixed, hierarchical, and highly interdependent: selecting an index type activates a different set of parameters, while the effect of each parameter depends on the remaining configuration, dataset, workload, and hardware environment [34, 37]. These complex interactions make it difficult for DBAs to reason about parameter effects or identify promising regions of the configuration space. Moreover, tuning experience gained from one deployment is difficult to transfer reliably to another [9, 37]. As a result, manual tuning typically relies on expert knowledge and iterative trial and error, with no guarantee of finding a high-performance configuration that satisfies the target recall constraint.

To reduce reliance on manual trial and error, automated tuning methods have been proposed for vector databases. Search-based methods explore a predefined set of candidate configurations and select the best observed one [36, 40]. Bayesian optimization (BO)-based methods build a surrogate model from previous evaluations and use it to select promising configurations for subsequent trials [37, 39]. Reinforcement learning (RL)-based methods instead learn a configuration policy through repeated interaction with the database environment [3, 9]. LLM-assisted DBMS tuners also exploit textual knowledge and workload context to restrict the search space or recommend complete configurations [10, 15].

Despite substantial progress in automated tuning, existing approaches still face two key limitations in practical vector database

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Table 1: Generalizability of vector database tuners.

Method	Tuning scope	Index families	Database engines	Offline dependence	Adaptation overhead
CHAT [5]	I+Q	HNSW	Multiple	Yes	Medium
PGTuner [9]	I+Q	Graph	Limited	Yes	Very high
VDTuner [37]	S+I+Q	Multiple	Milvus	No	High
EIGEN	S+I+Q	Multiple	Multiple	No	Low

S: system knobs; I: index configurations; Q: search parameters.

deployments: ① **low tuning efficiency** and ② **limited generalizability**. Figure 1 illustrates both issues on the Tiny5M dataset. On Milvus [35], existing methods require substantially longer tuning time than our approach to obtain competitive configurations (e.g., 506.0 hours for GridSearch vs. 5.4 hours for ours). When moving to pgvector [26], the gap becomes even larger: some methods require more tuning time while achieving lower QPS (e.g., VDTuner requires 12× the tuning time of our approach yet achieves 52.5% lower QPS), whereas PGTuner [9] cannot support the engine.

The *efficiency* issue stems from two factors: *the cost of each configuration evaluation* and *the number of evaluations required*. Existing methods evaluate candidate configurations directly on the full database, repeatedly rebuilding indexes and replaying the target workload, which makes each evaluation expensive [5, 36, 37]. At the same time, many evaluations are required to find a high-quality configuration: search-based methods explore numerous candidates. For example, GridSearch evaluates 4068 configurations for each dataset on Milvus. BO- and RL-based methods rely on repeated sampling and performance feedback to update their optimization models, often requiring hundreds of iterations. Together, expensive evaluations and a large number of iterations lead to prohibitively long end-to-end tuning time.

The *generalizability* limitations of existing tuners span several dimensions, as summarized in Table 1. First, their tuning scope and index-family coverage are often restricted: CHAT [5] and PGTuner [9] focus on index and search parameters and are specialized to HNSW or graph-based indexes, whereas VDTuner [37] provides broader coverage. Second, only CHAT [5] supports multiple backends, while PGTuner [9] has limited engine support and VDTuner [37] is designed for Milvus. Finally, existing methods may depend on offline components or incur substantial adaptation overhead when applied to a new deployment. Together, these limitations make it difficult to reuse the same tuning procedure effectively across heterogeneous environments.

To address these limitations, we propose **EIGEN**, an **Efficient** and **GENeralizable** framework for end-to-end vector database tuning. For tuning efficiency, EIGEN constructs multiple Mini-DBs through locality-aware stratified sampling and tunes them in parallel. Unlike relying on a single reduced dataset, it checks candidates across independently sampled views and ranks their performance and stability before full-database validation. It further introduces CALM, a constraint-aware LLM-guided optimizer that uses knob semantics and measured feedback to allocate the search budget. For generalizability, EIGEN separates engine-specific configuration descriptions and execution interfaces from its core tuning procedure, allowing the optimizer to be reused across different database engines and configuration spaces.

In summary, we make the following contributions:

- We present **EIGEN**, an efficient and generalizable framework for end-to-end vector database tuning. It jointly tunes system knobs, index configurations, and search parameters to maximize query throughput under user-specified recall constraints.
- We combine locality-aware sampling, cross-Mini-DB feasibility checks, and stability-aware ranking in a Mini-DB-based tuning strategy. Configuration evaluations operate on smaller databases, while final acceptance depends on the original database.
- We develop CALM, a constraint-aware LLM-guided optimizer with conditional search regions, explicit feasibility-weighted acquisition, and deterministic proposal validation. Optimizer ablations examine the contributions of region partitioning and iterative measurement feedback.
- We design a reusable tuning architecture that decouples database-specific configuration descriptions and execution interfaces from the core tuning procedure, enabling adaptation across vector databases, workloads, and hardware environments without task-specific offline training. Experiments across multiple databases characterize configuration quality, tuning time, and the contributions of the sampling and optimization components.

2 BACKGROUND AND RELATED WORK

Vector Databases. Vector databases are designed to support efficient similarity search over high-dimensional vectors using approximate nearest-neighbor (ANN) indexes. A variety of vector database systems have been developed with different architectures and design choices. Rather than exhaustively covering all available systems, we focus on three representative and commonly used systems: Milvus [35], Qdrant [28], and pgvector [26]. Milvus and Qdrant are purpose-built vector database systems with specialized indexing and data-management components, whereas pgvector integrates vector search into the general-purpose PostgreSQL DBMS. These architectural differences result in substantially different configuration interfaces and execution behaviors.

Tuning Space. Despite their architectural differences, tunable configurations in vector databases can be broadly organized into three layers. *System knobs* control resource allocation and execution behavior, such as memory management, parallelism, and background maintenance. *Index configurations* determine the index family and its construction parameters; for example, HNSW exposes M and $efConstruction$, whereas IVF-based indexes expose parameters such as $nlist$ and compression settings. *Search parameters* control query-time exploration, such as ef for HNSW and $nprobe$ for IVF. The concrete tuning space is engine-dependent: Milvus [35] supports multiple HNSW-, IVF-, and SCANN-based indexes, Qdrant [28] combines HNSW and quantization with segment and storage configurations, while pgvector [26, 27] exposes vector-index parameters together with PostgreSQL system knobs.

Vector Database Tuning. Vector database tuning shares the same general goal as conventional DBMS tuning: selecting configurations that improve workload performance based on runtime feedback. Traditional database tuning primarily adjusts system knobs to optimize throughput or latency while query semantics and result correctness remain unchanged. Vector database tuning, however, differs in two important aspects. First, ANN search deliberately

trades retrieval quality for efficiency [12, 20, 35], so index and search configurations can simultaneously affect both QPS and Recall@K. Given a configuration space C and target recall ρ , we seek

$$c^* = \arg \max_{c \in C} \text{QPS}(c) \quad \text{s.t.} \quad \text{Recall@K}(c) \geq \rho. \quad (1)$$

Second, its tuning space spans system, index, and search layers and is hierarchical, as each index family activates different parameters.

These differences also make existing tuners difficult to reuse. Such methods learn configuration–performance relationships specific to configuration spaces, workloads, and environments. Changes in databases can alter parameters and semantics, while workload or hardware changes can alter their performance effects. Thus, transfer requires additional measurements or retraining.

Search-based Tuning. Search-based methods directly evaluate candidate configurations and retain the best observed ones. GridSearch [36] discretizes parameter domains and enumerates their Cartesian product. It provides a strong reference for tuning quality when the grid is sufficiently dense, but the number of evaluations grows rapidly with the size and dimensionality of the configuration space. RandomSearch reduces this cost by evaluating only a sampled subset of configurations, but can miss high-quality regions in large heterogeneous spaces. For vector databases, CHAT [5] exploits HNSW-specific structural properties and resource-feasibility pruning to reduce unnecessary evaluations. Its specialization enables effective HNSW tuning across multiple backends, but does not directly extend to other ANN families or system-level knobs.

BO-based Tuning. Bayesian optimization (BO) models configuration performance with a surrogate and uses an acquisition function to select promising configurations for subsequent evaluation [23, 34, 39]. Compared with exhaustive search, BO can substantially reduce the number of physical evaluations by exploiting observations collected during tuning. VDTuner [37] applies multi-objective BO to vector database tuning and jointly explores index and search parameters while considering the trade-off between efficiency and recall. However, its tuning space and implementation are tailored to Milvus, requiring non-trivial adaptation for engines with different parameter spaces and execution interfaces.

Learning-based Tuning. Reinforcement learning (RL)-based methods learn configuration policies from performance feedback [17, 38]. These methods can exploit experience collected during tuning to navigate large configuration spaces more efficiently. For vector indexes, PGTuner [9] combines a pre-trained performance predictor with an RL-based configuration recommender. The predictor reduces physical evaluations by estimating configuration performance, while the recommender adapts configurations to a target dataset. However, its tuning scope is restricted to proximity-graph indexes such as HNSW, and applying the learned models to new settings introduces training-data collection and adaptation costs. More importantly, learned components rely on engine-specific features that are unavailable after switching database engines.

LLM-assisted Tuning. DB-BERT [33] uses a pre-trained language model to extract tuning hints from database manuals. GPTuner [15] uses LLMs to extract and consolidate domain knowledge, select knobs, refine their ranges, and guide coarse-to-fine Bayesian optimization. λ -Tune [10] generates complete configurations from workload and system context, then controls their evaluation and

reconfiguration costs. These studies demonstrate the potential of language models for database tuning. Our contribution is therefore not the use of an LLM itself, but its integration into a constraint-aware tuning workflow for vector databases. Specifically, CALM searches the conditional system–index–search space under an explicit recall constraint.

Sampling for Lower-Cost Tuning. Evaluating configurations on reduced data is also established practice. For example, FLANN [21] can select ANN algorithms and parameters using a fraction of the input dataset before applying the selected configuration at full scale. Our construction builds on stratified sampling [6] and metric-compatible locality-sensitive hashing [4, 7]. The distinction is the combination of locality-aware coverage, independent sampled views, and cross-view feasibility and stability screening for end-to-end vector database configurations. We assess this combination against uniform sampling and mean-only ranking in Section 6.5.2.

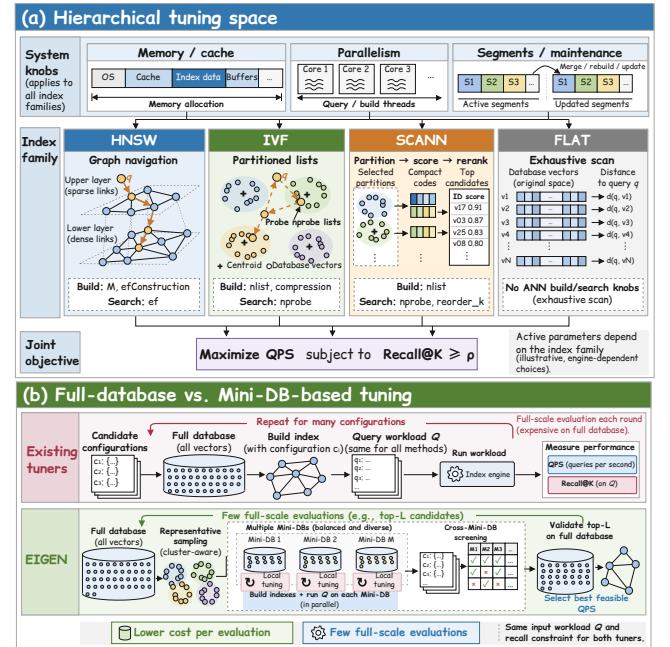


Figure 2: Hierarchical vector database tuning space and full-database versus Mini-DB-based configuration evaluation.

Summary. Figure 2 highlights two characteristics that motivate our design. First, vector database tuning requires navigating a hierarchical configuration space spanning system, index, and search layers. Second, reliable performance feedback is commonly obtained through physical index construction and workload execution on the full database, making each evaluation costly. EIGEN addresses these two aspects by combining a reusable, semantics-guided optimizer with representative Mini-DBs that move most configuration evaluations away from the full database.

3 EIGEN OVERVIEW

Given a vector database, a query workload, and a target recall constraint, EIGEN jointly tunes system knobs, index configurations, and search parameters to maximize QPS while satisfying the recall requirement. As shown in Figure 3, the workflow consists of three phases: *data abstraction*, *parallel optimization*, and *global selection*.

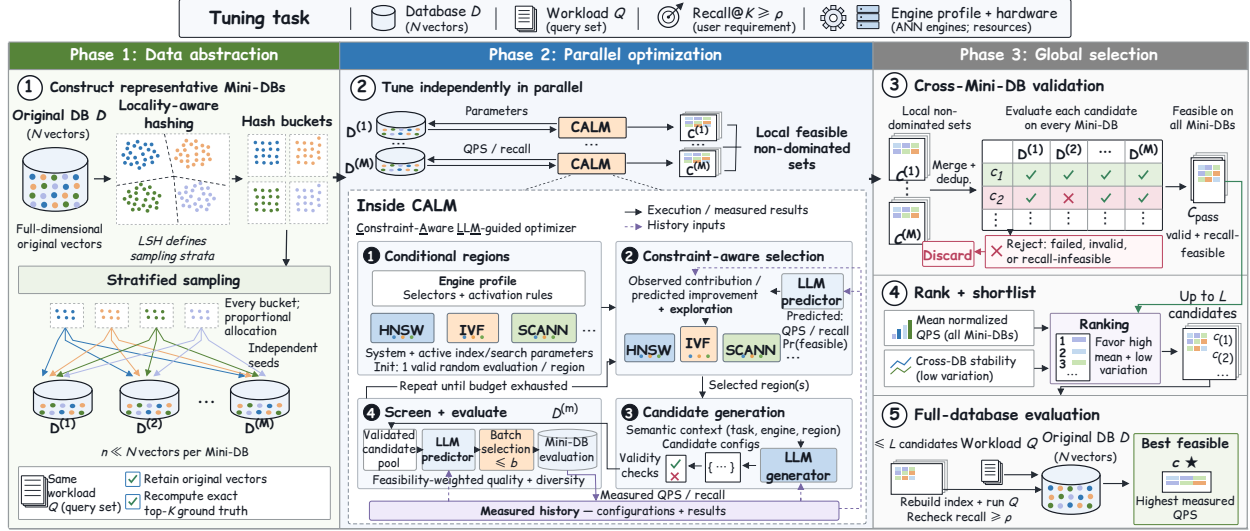


Figure 3: End-to-end workflow of EIGEN: data abstraction with representative Mini-DBs, parallel optimization with CALM, and global selection with final full-scale evaluation.

3.1 Data Abstraction

The first phase constructs multiple representative Mini-DBs from the original database (Stage ①). EIGEN uses locality-aware stratified sampling to generate smaller databases that preserve the local structure of the original data. Because sampling changes the available nearest neighbors, exact Top- K ground truth is recomputed for each Mini-DB, making each one a self-contained environment for configuration evaluation.

Mini-DBs are introduced to reduce the cost of each configuration evaluation. Direct evaluation on the full database can be expensive because each trial requires index construction and workload execution over the complete dataset. By shifting most evaluations to much smaller databases, Mini-DBs substantially reduce this cost, while multiple independently sampled Mini-DBs reduce reliance on any single sample. Section 4 provides further details.

3.2 Parallel Optimization

The second phase tunes the Mini-DBs independently and in parallel under the same objective and recall constraint (Stage ②). Each Mini-DB is optimized by CALM, our constraint-aware LLM-guided optimizer. CALM uses configuration semantics, validity rules, historical evaluations, and measured feedback to identify promising configurations. Each tuning instance produces a local candidate set.

CALM is designed to reduce the number of physical evaluations. The vector database tuning space is large, hierarchical, and engine-specific, making exhaustive search inefficient and purely data-driven optimization costly. By combining semantic guidance with measured database feedback, CALM focuses the evaluation budget on promising regions while retaining exploration. Section 5 provides further details.

3.3 Global Selection

The final phase combines the candidate sets produced by all Mini-DBs. EIGEN first merges and deduplicates the candidates and evaluates each one across all Mini-DBs (Stage ③). Configurations violating the recall constraint are removed, and the remaining candidates

are ranked according to their average normalized performance and cross-Mini-DB stability (Stage ④). At most L candidates ($L = 5$ by default) are transferred to the original database for full-scale evaluation. The feasible configuration with the highest measured QPS is returned (Stage ⑤).

4 MINI-DB-BASED TUNING FRAMEWORK

This section presents our Mini-DB-based tuning framework, which moves most configuration evaluations to smaller sampled databases. As summarized in Algorithm 1, EIGEN constructs and tunes multiple Mini-DBs, cross-validates their candidate configurations, and transfers a short list to the original database for final validation.

4.1 Transfer Intuition

Our insight is that representative sampling can preserve coarse data geometry and query distribution, which influence the trade-offs among index structure, search breadth, and compression. Mini-DB evaluations can therefore help identify promising configurations even when absolute QPS changes with scale. This rationale is consistent with NVIDIA’s tuning guidance [22], which emphasizes matching local index scale and distribution in locally partitioned systems and preserving global vector and query distributions in partitioned systems. Accordingly, our locality-aware sampling maintains coverage across data regions and approximately preserves their relative populations, while retaining the original query workload.

We reuse configuration values directly and rebuild the corresponding indexes on the full database. In segment-based engines, growth can also increase the number of local indexes without proportionally increasing their individual sizes, making direct reuse plausible when local index characteristics remain comparable.

Our goal is to retain promising configurations in a shortlist without requiring exact preservation of neighborhoods or configuration rankings. Cross-Mini-DB screening reduces sensitivity to individual samples, while full-database validation selects the final feasible configuration under actual memory pressure and segment layout.

Algorithm 1 Mini-DB-Based Tuning Framework

Require: Database $\mathcal{D}$, queries $\mathcal{Q}$, distance metric d , recall threshold ρ ; M Mini-DBs of size n ; local evaluation budgets $B_1, \dots, B_M$; transfer budget L

Ensure: Best feasible configuration

```
1:  $\mathcal{B} \leftarrow \text{HashBucketize}(\mathcal{D}, n, d)$ 
2: for  $m = 1, \dots, M$  do
3:    $\mathcal{D}^{(m)} \leftarrow \text{StratifiedSample}(\mathcal{D}, \mathcal{B}, n)$  with independent seed
4:   Recompute exact top- $K$  ground truth on  $\mathcal{D}^{(m)}$ 
5: end for
6: for  $m = 1, \dots, M$  in parallel do
7:    $C^{(m)} \leftarrow \text{Tune}(\mathcal{D}^{(m)}, \mathcal{Q}, \rho, B_m)$  {local feasible non-dominated candidates}
8: end for
9:  $C \leftarrow \bigcup_{m=1}^M C^{(m)}$ 
10: Deduplicate  $C$  by configuration
11: Evaluate every  $c \in C$  on all Mini-DBs
12: Let  $C_{\text{pass}}$  contain candidates with valid, successful measurements satisfying  $\text{Recall@K} \geq \rho$  on every Mini-DB
13: for each  $c \in C_{\text{pass}}$  do
14:   Compute  $S(c)$  from mean performance and cross-Mini-DB stability
15: end for
16:  $C_L \leftarrow \text{TopL}(C_{\text{pass}}, S, L)$ 
17: return  $\text{FullScaleValidate}(C_L, \mathcal{D}, \mathcal{Q}, \rho)$ 
```

4.2 Locality-Aware Mini-DB Construction

Given a database $\mathcal{D}$ containing N vectors, we construct M Mini-DBs, each containing $n \ll N$ original vectors, through locality-aware stratified sampling. We partition $\mathcal{D}$ using locality-sensitive hashing compatible with the workload distance metric d : random-hyperplane LSH for cosine similarity [4] and Gaussian p -stable LSH for L_2 distance [7]. Hashing only defines the sampling strata; the original high-dimensional vectors remain unchanged for index construction, ground-truth computation, and query execution.

To determine partition granularity, we consider progressively longer hash signatures and select the longest available prefix yielding at most n non-empty buckets. We allocate one sample to each bucket and distribute the remaining budget proportionally to bucket populations [6], subject to each bucket’s remaining capacity. Within each bucket, vectors are sampled uniformly without replacement. All Mini-DBs share the same partition, while independent sampling seeds generate different samples.

Sampling also changes the set of vectors available to each query. Thus, the full-database ground truth cannot be reused directly. For every Mini-DB, we recompute the exact top- K neighbors of the same query workload using the corresponding workload metric. Each Mini-DB therefore forms a self-contained evaluation environment in which QPS and Recall@K can be measured consistently.

4.3 Cross-Mini-DB Aggregation and Full-Database Validation

After all Mini-DBs are constructed, they are tuned independently and in parallel using the optimizer in Section 5. For Mini-DB m , the tuner returns a local candidate set $C^{(m)}$ containing the feasible non-dominated configurations retained by the optimizer. We merge

these sets and deduplicate identical configurations to obtain

$$C = \bigcup_{m=1}^M C^{(m)}.$$

To reduce sample-specific bias, we evaluate every $c \in C$ on all Mini-DBs, discarding candidates if any evaluation fails, yields invalid measurements, or violates the recall constraint.

Among the surviving candidates, we consider both performance and cross-Mini-DB stability. For each candidate c , let $\hat{f}_m(c) \in [0, 1]$ denote its QPS on Mini-DB m , min-max normalized over the surviving candidate pool on that Mini-DB. We compute

$$S(c) = \text{Norm}\left(\text{mean}_m \hat{f}_m(c)\right) + \lambda \left[1 - \text{Norm}\left(\text{std}_m \hat{f}_m(c)\right)\right], \quad (2)$$

Here Norm denotes min-max normalization across surviving candidates, with a constant input mapped to zero; std is the population standard deviation across Mini-DBs. The weight λ controls the contribution of stability and defaults to 1. The score favors high mean normalized QPS and low variability across Mini-DBs.

We rank the surviving candidates by $S(c)$ and select up to L configurations to form C_L ($L = 5$ by default). These configurations are evaluated on the original database using the same workload, and the recall-feasible configuration with the highest measured QPS is returned.

For a completed run with one measurement per scheduled evaluation, let C_{pass} be the candidate set surviving cross-Mini-DB screening. The total number of physical configuration evaluations is

$$E_{\text{total}} = \sum_{m=1}^M B_m + M|C| + \min\{L, |C_{\text{pass}}|\}. \quad (3)$$

The three terms count local search, cross-Mini-DB re-evaluation, and full-database validation, respectively. The implementation re-evaluates the entire deduplicated candidate pool on every Mini-DB, including each candidate’s originating view. Local budgets count invoked database evaluations, including failed measurements; rejected proposals and LLM calls are not database evaluations.

4.4 Design Validation

We examine ranking fidelity and recall transfer, and use ablations to assess the end-to-end contributions of Mini-DB tuning, locality-aware sampling, and stability-aware aggregation. Further analyses assess sensitivity to Mini-DB number and size and quantify runtime overhead. These evaluations are presented in Section 6.5.1, 6.5.2, 6.6.1, and 6.7.

5 CONSTRAINT-AWARE LLM-GUIDED OPTIMIZER

This section presents CALM, a constraint-aware LLM-guided optimizer for tuning each Mini-DB. Prior studies have demonstrated the use of LLMs in optimization [18, 30]. CALM uses semantic candidate generation and measured feedback to search conditional vector database configuration spaces under a recall constraint. Given a Mini-DB, its workload, an engine profile, target recall ρ , and local evaluation budget B_m , CALM searches for configurations that maximize QPS subject to $\text{Recall@K} \geq \rho$. As shown in Figure 3, CALM follows four stages: conditional region construction, constraint-aware

region selection, LLM-based candidate generation, and surrogate screening with physical Mini-DB evaluation.

Semantic guidance with constraint-aware control. The design combines the LLM’s ability to use task semantics with explicit constraints and empirical validation. The LLM relates parameter meanings, index characteristics, workload properties, and hardware conditions to measured outcomes, providing a task-informed search prior for targeted configuration changes. The surrounding framework controls how proposals are generated, evaluated, and retained: conditional regions expose applicable parameters and dependencies, validity checks reject illegal configurations, and constraint-aware selection and screening direct physical evaluations toward promising candidates under the recall requirement. Physical Mini-DB evaluations then measure recall and QPS, determining candidate feasibility and performance while providing feedback to refine subsequent proposals. Together, these mechanisms make semantic guidance subject to explicit validity checks and measured evidence throughout the search.

5.1 Tuning Workflow

Stage 1: Conditional Search Regions. CALM constructs regions from the engine profile’s selectors and activation rules, fixing selector values and retaining only applicable parameters in each region. For example, selecting HNSW activates its graph parameters, whereas selecting IVF activates its partitioning parameters; system parameters remain available wherever applicable. This decomposition separates structurally different choices while preserving joint tuning within each region. Initialization evaluates one valid random configuration per region, and all count toward B_m .

Stage 2: Constraint-Aware Region Selection. Stages 2 and 4 share a history-conditioned LLM predictor. In a separate prediction request, it receives the task context, configuration schema, measured history, and candidate configurations, and returns predicted QPS and recall means and standard deviations together with the probability of satisfying the constraints. These predictions guide evaluation allocation; physical measurements determine actual feasibility.

Let $\mathcal{P}_t$ be the measured feasible non-dominated archive in QPS and recall. Retaining recall as an auxiliary search coordinate preserves alternative speed-accuracy trade-offs for exploration and cross-Mini-DB validation. Let $n_t(R)$ count successful measurements in region R . For a sufficiently explored region, its contribution is $\Delta HV_t(R) = HV(\mathcal{P}_t) - HV(\mathcal{P}_t \setminus R)$, computed with the same coordinate normalization and reference point. For an underexplored region, random probes $c \in \mathcal{U}_t(R)$ provide predicted QPS mean $\hat{q}_t(c)$, standard deviation $\hat{\sigma}_{q,t}(c)$, and feasibility probability $\hat{p}_t(c)$. With incumbent feasible QPS q_t^* (zero if unavailable), define

$$g_t(c) = \frac{[\hat{q}_t(c) + \beta_{\text{probe}} \hat{\sigma}_{q,t}(c) - q_t^*]_+}{\max\{1, |q_t^*|\}}, \quad (4)$$

where $[x]_+ = \max\{0, x\}$. The region score is

$$s_t(R) = v_t(R) + \frac{\eta_{\text{region}}}{\sqrt{n_t(R) + 1}}, \quad (5)$$

$$v_t(R) = \begin{cases} \Delta HV_t(R), & n_t(R) \geq h, \\ \max_{c \in \mathcal{U}_t(R)} \hat{p}_t(c) g_t(c), & \text{otherwise.} \end{cases}$$

An empty probe set contributes zero. The score combines observed contributions from established regions with predicted feasible improvement from less explored ones; the exploration bonus discourages premature concentration. The LLM probabilities are heuristic estimates. Among available regions $\mathcal{R}$, selection uses

$$\Pr_t(R) = \frac{\epsilon_{\text{region}}}{|\mathcal{R}|} + (1 - \epsilon_{\text{region}}) \frac{s_t(R)}{\sum_{R' \in \mathcal{R}} s_t(R')}, \quad (6)$$

with uniform selection if the score sum is zero. Multiple regions are sampled without replacement. Table 2 summarizes the default settings.

Stage 3: LLM Candidate Generation. For each selected region, the LLM receives the workload, hardware and runtime context, engine schema, active parameters, objectives, recall constraint, and recent successful evaluations. It uses this context to propose unseen configurations in structured JSON, relating parameter changes to the current performance and feasibility bottlenecks. A deterministic validator checks parameter names, types, ranges, activation rules, and dependencies, and removes duplicates. For example, the Milvus profile requires $n_{\text{probe}} \leq n_{\text{list}}$ and HNSW search width at least K . Valid random proposals fill any remaining generation shortfall. Independently, each proposal slot uses random sampling with probability p_{proposal} to retain exploration beyond the LLM’s preferred configurations.

Stage 4: Surrogate Screening and Mini-DB Evaluation. The LLM predictor introduced in Stage 2 estimates the metrics and feasibility of the validated candidate pool. Screening reserves physical evaluations for promising candidates while encouraging diversity within each batch. It uses optimistic QPS and recall coordinates $\hat{\mu} + \kappa_{\text{batch}} \hat{\sigma}$, normalized jointly with the measured feasible archive. For candidate c , let $\widehat{\Delta HV}_t(c)$ be its predicted hypervolume improvement and $\bar{z}_t(c)$ its mean normalized coordinate. Its quality weight is

$$w_t(c) = \hat{p}_t(c) \left[\widehat{\Delta HV}_t(c) + 1_{\{\mathcal{P}_t = \emptyset\}} \bar{z}_t(c) \right]. \quad (7)$$

The second term provides a quality signal before a feasible archive is available. Given an already selected batch $\mathcal{S}$, the next candidate is scored by

$$a_t(c \mid \mathcal{S}) = \text{Norm}(w_t(c)) + \gamma \min_{s \in \mathcal{S}} d(c, s), \quad (8)$$

where d combines range-normalized numeric differences with categorical and activation mismatches over applicable parameters. The diversity term favors candidates that provide different search directions instead of spending the batch on near-duplicates. Each selection is uniform at random with probability p_{batch} and otherwise maximizes this score. Only the selected configurations are physically evaluated, subject to the remaining local budget.

Evaluation History and Local Candidates. Measured evaluations update the history used in subsequent region selection, candidate generation, and surrogate prediction. At termination, CALM returns its deduplicated feasible non-dominated archive as $\mathcal{C}^{(m)}$ for cross-Mini-DB aggregation (Section 4.3).

5.2 Design Validation

Section 6.5.3 evaluates the CALM design choices by comparing the complete optimizer with variants that remove region partitioning

Table 2: Default settings of CALM.

Symbol	Meaning	Value
b_{init}	Initial evaluations per Mini-DB	$ \mathcal{R} $
h	Region observation threshold	3
u	Prediction probes per region	2
C	Proposals per round	12
b	Maximum evaluation batch size	4
β_{probe}	Probe optimism	0.2
η_{region}	Region exploration bonus	0.2
κ_{batch}	Batch optimism	0.2
ϵ_{region}	Uniform region selection	0.1
$p_{\text{proposal}}, p_{\text{batch}}$	Random proposals / batch selection	0.1 each
γ	Batch diversity weight	0.2

or iterative optimization, as well as with constrained Bayesian optimization. The experiments compare convergence and final QPS under different recall constraints. Section 6.6.2 further evaluates different LLM backbones and reasoning settings to examine the sensitivity of CALM to the underlying model.

6 EXPERIMENTAL EVALUATION

This section compares EIGEN with representative vector database tuners. The main results (Section 6.2) report tuning time and configuration quality. The generalizability study (Section 6.3) and drift experiments (Section 6.4) examine transfer across deployment settings and adaptation to data and query drift, respectively. Ablations (Section 6.5), sensitivity analysis (Section 6.6), and overhead analysis (Section 6.7) examine individual design choices and their costs.

6.1 Experimental Setup

Workloads. We use vector-db-benchmark [29] to generate workloads for six datasets, comprising five real-world vector datasets and the synthetic Geo-radius workload: Nytimes¹, Glove², GIST³, Tiny5M⁴, Msong⁵, and Geo-radius⁶ [2]. Table 3 summarizes the datasets; Tiny5M contains five million vectors and is the largest evaluated database.

Table 3: Evaluated Datasets.

Dataset	Database size	Queries	Dimension	Distance	Type
Nytimes	290,000	1,000	256	Cosine	Text
GloVe	1,183,514	10,000	100	Cosine	Text
GIST	1,000,000	1,000	960	L2	Image
Tiny5M	5,000,000	1,000	384	L2	Image
Msong	994,185	200	420	L2	Audio
Geo-radius	100,000	1,000	2,048	Cosine	Synthetic

Hardware and Software. We evaluate EIGEN primarily on Milvus v2.3.1, whose diverse index families and configurable system, index, and search parameters make it suitable for evaluating joint tuning across these layers. Qdrant v1.16.0 and pgvector v0.6.2 are used in the engine-generalizability study. Unless otherwise stated, each machine has a 16-core 3.60 GHz CPU and 64 GB of RAM. Mini-DB workers run concurrently on separate machines with identical

¹<https://ann-benchmarks.com/nytimes-256-angular.hdf5>

²<http://ann-benchmarks.com/glove-100-angular.hdf5>

³<http://ann-benchmarks.com/gist-960-euclidean.hdf5>

⁴<https://www.cse.cuhk.edu.hk/systems/hash/gqr/dataset/tiny5m.tar.gz>

⁵<https://www.cse.cuhk.edu.hk/systems/hash/gqr/dataset/msong.tar.gz>

⁶https://storage.googleapis.com/ann-filtered-benchmark/datasets/random_geo_1m.tgz

hardware specifications. For fair comparison, their individual optimization times are summed in the reported tuning time. The default LLM for both candidate generation and surrogate prediction is GPT-5.4 with medium reasoning effort. Section 6.6.2 examines other models and reasoning settings.

Baselines. EIGEN is compared with the following methods:

- **GridSearch** [36] sets the vector database according to predefined parameter configurations to search for the best one.
- **RandomSearch** uses Latin Hypercube Sampling (LHS) [19] to randomly sample a small number of parameter configurations to select the best sampled configuration.
- **VDTuner** [37] employs the Gaussian process regression and Constrained Bayesian optimization to auto-tune vector database.
- **PGTuner** [9] uses deep reinforcement learning to recommend proximity graph configurations.
- **CHAT** [5] is an HNSW-specific tuner that leverages the index structure for deterministic configuration search and early pruning of resource-infeasible configurations.

Table 4: Index types and corresponding parameters.

Supported Index	Description	Building & Searching Parameters
AUTOINDEX	Default configuration	N/A ; N/A
FLAT	Exhaustive approach	N/A ; N/A
HNSW [20]	Graph-based	M, efConstruction ; ef
IVF FLAT [12]	Quantization-based	nlist ; nprobe
IVF SQ8 [12]	Quantization-based	nlist ; nprobe
IVF PQ [13]	Quantization-based	nlist, m, nbits ; nprobe
SCANN [11]	Quantization-based	nlist ; nprobe, reorder k

Tuning Settings. As shown in Table 4, evaluated parameters for Milvus are 16-dimensional in total, including index type, 8 index parameters and 7 system parameters. For GridSearch, we discretize the range of each parameter to construct a predefined coarse grid comprising 4068 complete configurations. Following prior work [9], RandomSearch employs stratified LHS to sample 14 index-construction configurations for each index type, resulting in 98 configurations in total. For VDTuner, we set the tuning budget to 200 rounds, consistent with its original paper. As PGTuner and CHAT support only graph-based index tuning, parameters outside its supported search space retain their default values.

Unless otherwise stated, we use $M = 3$ Mini-DBs, a sampling ratio of 10%, $B_m = 20$ local physical configuration evaluations per Mini-DB, stability weight $\lambda = 1$, and transfer budget $L = 5$. For the Milvus search space, initialization uses seven evaluations per Mini-DB, leaving up to 13 for guided search. The resulting local budget is $\sum_{m=1}^M B_m = 60$. The remaining optimizer settings are listed in Table 2. Each tuning run uses one fixed recall threshold. Local search counts include initialization and failed database evaluations, but exclude LLM requests, unevaluated probes and proposals, and the subsequent cross-Mini-DB and full-database measurements counted separately in Eq. 3.

Evaluation Metric. Tuning effectiveness is the full-database QPS of the recommended configuration satisfying a given recall constraint. To examine tuning performance under progressively tighter accuracy constraints, we use seven recall targets: $\{0.85, 0.875, 0.90, 0.925, 0.95, 0.975, 0.99\}$. These targets sample the range from 0.85 to 0.975 at 0.025 intervals, with 0.99 added to cover near-exact retrieval.

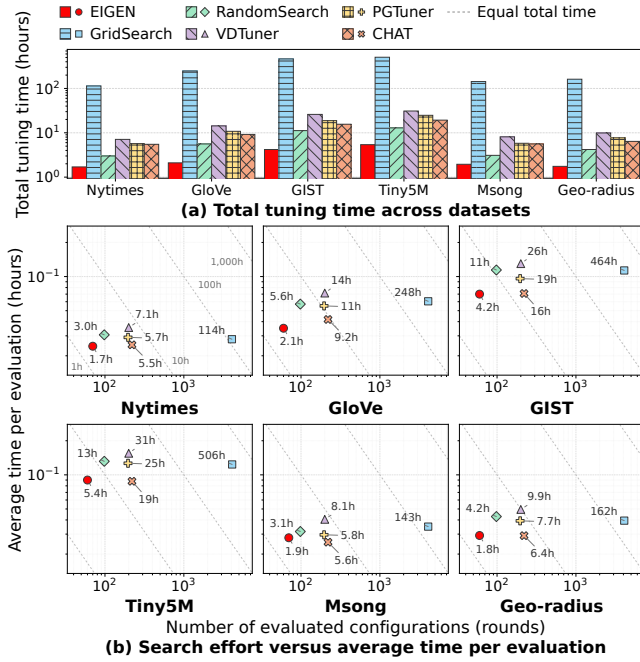


Figure 4: Tuning time across methods. (a) Total tuning time by dataset. (b) Search count and average time per counted evaluation. For EIGEN, we sum the tuning iterations and optimization time across all Mini-DBs for a fair comparison.

Tuning time is the aggregate time charged to a complete tuning run. For EIGEN, it includes Mini-DB construction, the sum of individual worker optimization times including LLM calls, cross-Mini-DB evaluation and ranking, and full-database validation. Concurrent worker activity is charged separately, so additional machines do not reduce the reported optimization cost. We apply this accounting consistently in tuning-time comparisons and the component breakdown. For PGTuner and CHAT, we follow their original reporting conventions and exclude offline pretraining.

6.2 Tuning Performance Evaluation.

We evaluate tuning performance in terms of *tuning efficiency* and *configuration quality* across six different benchmarks. The main comparisons report a single run per setting. We assess run-to-run variability through repeated experiments in Section 6.5.2, where the QPS of configurations recommended by EIGEN exhibits a coefficient of variation of only 0.45% under the tested setting.

6.2.1 Tuning Efficiency Comparison. Tuning efficiency is a key advantage of EIGEN. As reported in Figure 4(a), EIGEN reduces tuning time by 37.10%-99.15% compared with existing methods. The reported average speedup over the next-fastest method is 2.25 $\times$ across the evaluated datasets. GridSearch is a different reference: it is competitive in QPS (Section 6.2.2) but expensive in tuning time. Its reported times are 67.00 $\times$, 118.00 $\times$, 110.43 $\times$, 93.70 $\times$, 73.38 $\times$, and 92.34 $\times$ those of EIGEN on NYTimes, GloVe, GIST, Tiny5M, Msong, and Geo-radius, respectively.

Figure 4(b) helps explain this advantage: EIGEN uses fewer configuration evaluations (x-axis) and incurs a lower average cost per evaluation (y-axis). This efficiency stems from two complementary

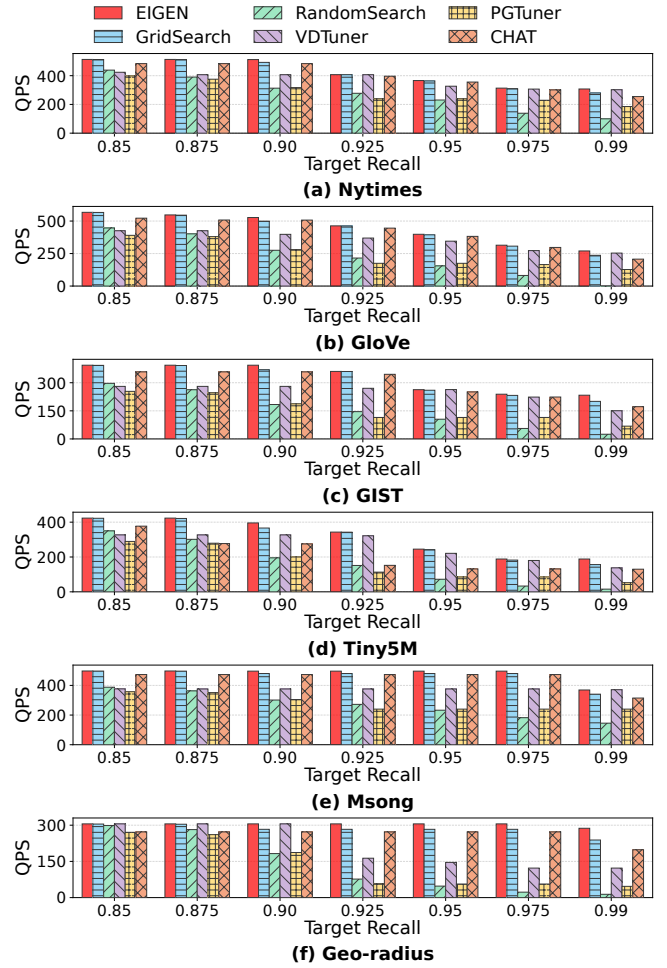


Figure 5: Full-database QPS of recommended configurations at the seven target recall thresholds.

mechanisms: Mini-DBs reduce the data-loading, index-construction, and query-execution costs of candidate evaluation, while the region-wise, constraint-aware optimizer focuses exploration on promising feasible configurations. Together, these mechanisms reduce both search effort and per-evaluation cost, contributing to the overall tuning-time savings.

6.2.2 Tuning Effect Comparison. In addition to tuning efficiently, EIGEN recommends high-quality configurations. We compare the QPS of recommended configurations across datasets and recall targets in Figure 5, where EIGEN achieves the best or comparable QPS. Relative to GridSearch, the strongest evaluated baseline in observed QPS, the average improvements are 1.98%, 2.53%, 3.12%, 3.48%, 2.89%, and 7.13% on NYTimes, GloVe, GIST, Tiny5M, Msong, and Geo-radius, respectively. A finite grid covers only a discretized subset of the mixed configuration space, whereas guided search can evaluate configurations outside that grid. The average improvement over RandomSearch is 94.53%. CHAT explores HNSW-specific parameters more thoroughly but does not jointly tune system knobs; its QPS is 12.58% lower on average than EIGEN. PGTuner also has a narrower, index-focused tuning scope, and EIGEN achieves 88.85%

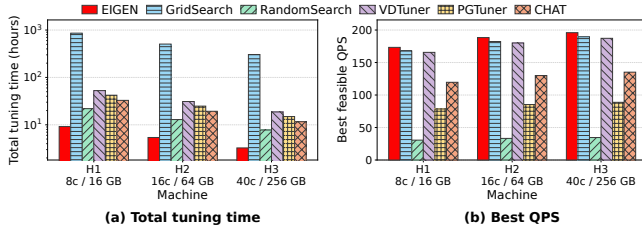


Figure 6: Hardware adaptability of different tuning methods. (a) Tuning time under different hardware settings. (b) Maximum feasible QPS under those settings.

higher QPS on average. The corresponding improvement over VDTuner is 32.72%. This advantage stems from EIGEN’s region-wise, constraint-aware search: partitioning the heterogeneous configuration space enables more focused exploration, while recall-aware candidate selection directs evaluations toward configurations that offer higher QPS while satisfying the target recall.

6.3 Generalizability Study

We evaluate the generalizability of EIGEN across hardware environments and database engines using Tiny5M, our largest dataset. We select a recall target of 0.975 to test generalizability under a stricter accuracy constraint than the 0.95 reference used in the subsequent drift and ablation studies.

6.3.1 Hardware. As detailed in Section 6.1, our primary evaluations are conducted on a server equipped with a 16-core CPU and 64 GB of RAM. To evaluate the adaptability of EIGEN across different hardware environments, we additionally deploy the database on two machines with substantially different resource capacities: one equipped with 8 CPU cores and 16 GB of RAM, and the other with 40 CPU cores and 256 GB of RAM. We denote these three hardware environments, in ascending order of resource capacity, as H1, H2, and H3, respectively. Figure 6 reports the experimental results.

Figure 6(a) shows that EIGEN has the shortest reported tuning time in all three settings, achieving an average 2.39× speedup over the next-fastest method. Thus, its tuning-time advantage persists from the resource-constrained H1 setting to the larger H3 setting. Figure 6(b) shows that, although absolute QPS changes with hardware capacity, EIGEN achieves the best or comparable QPS in each setting. Its average QPS improvements over the baselines range from 3.3% to 466.4%. Together, these results show that EIGEN tunes faster while maintaining high configuration quality across all three hardware settings, demonstrating its adaptability to different hardware environments. By incorporating the current hardware profile, EIGEN enables the LLM to propose configurations that account for available resources and hardware-dependent parameter ranges.

6.3.2 DB Engine. Our primary experiments are conducted on Milvus v2.3.1. We further evaluate it on Qdrant v1.16.0 and pgvector v0.6.2 to evaluate whether EIGEN can generalize across different vector database engines. As summarized in Table 5, Milvus, Qdrant and pgvector expose substantially different tuning spaces. For EIGEN, adapting to new database engines only requires specifying the corresponding tuning space and database execution interface; the underlying optimization framework remains unchanged and requires no pre-training or retraining. We compare the tuning

Table 5: Tuning spaces of vector database engines.

Engine	Category	Tunable Parameters
Milvus	Index	AUTO/FLAT; HNSW (M, efConstruction); IVF-FLAT/SQ8 (nlist); IVF-PQ (nlist, m, nbits); SCANN (nlist)
	Search System	HNSW (ef); IVF-* (nprobe); SCANN (nprobe, reorder_k, gracefulTime, minSegmentSizeToEnableIndex, segment_maxSize, segment_sealProportion, insertBufSize, autoHandoff, autoBalance)
Qdrant	Index	HNSW (m, ef_construct); quantization (quantile, compression, encoding, always_ram)
	Search System	hnsf_ef, exact, rescore, oversampling, ignore_full_scan_threshold, max_indexing_threads, on_disk_payload_m, inline_storage, on_disk_payload, default_segment_number, max_segment_size, mmap_threshold, indexing_threshold, flush_interval_sec, max_optimization_threads
pgvector	Index	HNSW (m, ef_construction); IVFFlat (lists)
	Search System	HNSW (ef_search); IVFFlat (probes) shared_buffers, effective_cache_size, maintenance_work_mem, max_wal_size, work_mem, effective_io_concurrency, random_page_cost, max_parallel_workers, max_parallel_maintenance_workers

effectiveness and efficiency of different methods. The results are reported in Figure 7.

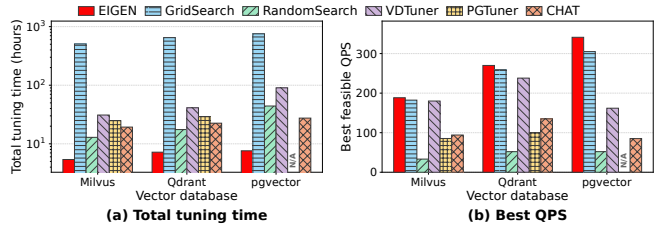


Figure 7: Database adaptability of different tuning methods. (a) Tuning time under different vector databases. (b) Maximum feasible QPS under those databases.

Figure 7 shows that EIGEN maintains competitive tuning time and configuration quality across the evaluated engines. On Qdrant, it completes tuning in 7.2 hours, 2.43× faster than the next-fastest method, while achieving 4.2% higher QPS than the next-best configuration. Together with its results on Milvus, this shows that the same optimization procedure remains effective across two engines with different configuration spaces.

pgvector provides a further test. It is an extension to the traditional relational database PostgreSQL and therefore exposes a substantially different tuning interface. PGTuner cannot be applied through the evaluated pgvector interface because its required internal distance-count metric is unavailable, and is therefore marked “N/A” in Figure 7. CHAT can be applied but does not jointly tune PostgreSQL system knobs and vector-index parameters; it achieves lower QPS in this setting. In contrast, EIGEN attains the highest feasible QPS and the lowest reported tuning time on pgvector. Its engine profile supplies PostgreSQL and pgvector configuration semantics to the same optimizer, while Mini-DBs reduce the cost of search evaluations in this more expensive deployment.

6.4 Performance Comparison in Drift Scenarios

In real-world deployments, vector databases are rarely static and may experience distributional changes over time. Such drift can lead to performance degradation of previously optimal configurations. In this section, we simulate two common types of drift, *data drift* and

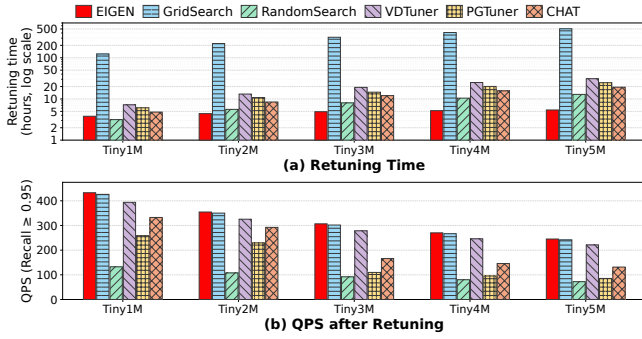


Figure 8: Tuning performance as the database grows. (a) Retuning time. (b) Maximum feasible QPS across increasing database sizes.

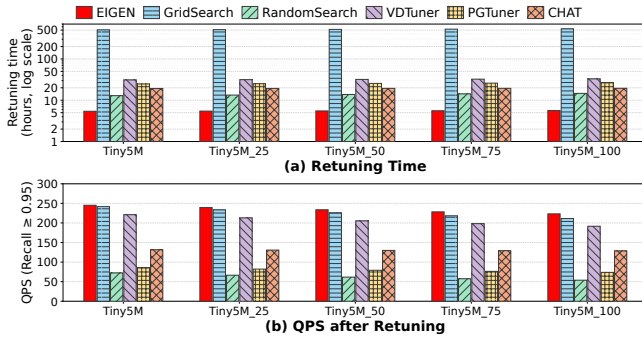


Figure 9: Tuning performance under query drift. (a) Retuning time. (b) Maximum feasible QPS across increasing query-drift levels.

query drift, to evaluate the robustness and adaptability of EIGEN. We fix the recall target at 0.95, a high-recall reference point within the evaluated range, to compare adaptation across drift levels under a consistent accuracy requirement.

6.4.1 Data Drift. In this section, we extract subsets containing 1M, 2M, 3M, and 4M vectors from Tiny5M, denoted as Tiny1M, Tiny2M, Tiny3M, and Tiny4M, respectively. Together with the original Tiny5M dataset, these datasets represent a gradually increasing database size. We therefore perform tuning sequentially as the database grows from Tiny1M to Tiny5M.

Figure 8(b) reports QPS on the growing Tiny datasets. EIGEN achieves the best or comparable measured QPS across the evaluated sizes. Figure 8(a) compares retuning time. On Tiny1M, the speedup over the next-fastest method is 1.26 $\times$. From Tiny1M to Tiny5M, the tuning time of EIGEN grows by 1.42 $\times$, compared with 4.12 $\times$ on average for the baselines, and the speedup on Tiny5M is 2.39 $\times$. This favorable scalability primarily stems from the Mini-DB design. When the original database is relatively small, the reduction in evaluation cost brought by Mini-DBs is less pronounced. As the database size grows, however, evaluating configurations on the full database becomes expensive, while Mini-DBs keep the per-configuration evaluation cost lower. As a result, the efficiency advantage of EIGEN becomes more significant at larger scales.

6.4.2 Query Drift. To simulate continuous query drift, we perturb the queries in Tiny5M by adding Gaussian noise at levels of 25%, 50%, 75%, and 100%, resulting in four drifted workloads denoted as

Tiny5M_25, Tiny5M_50, Tiny5M_75, and Tiny5M_100, respectively. Accordingly, we perform tuning from Tiny5M_25 to Tiny5M_100.

Figure 9(a) compares the tuning time of different methods under query drift. EIGEN remains the most tuning-efficient method across all query-drift scenarios, showing that its efficiency advantage is preserved even under continuously changing query distributions. Figure 9(b) further compares QPS after retuning the perturbed workloads. EIGEN consistently achieves the best tuning performance across all drift levels, demonstrating its ability to maintain high-quality tuning results as the query workload evolves.

6.5 Ablation Study

We conduct ablation experiments to examine the two key components of EIGEN: the Mini-DB framework and CALM. Unless otherwise stated, all experiments use Milvus and Tiny5M on the testbed described in Section 6.1, with recall fixed at 0.95 to compare component contributions under the same high-recall requirement used in the drift study.

Table 6: Fidelity of Mini-DBs in preserving the configuration-performance landscape. Values are mean $\pm$ sample standard deviation over three independently constructed Mini-DBs.

Dataset	Sampling	Rank Corr. $\uparrow$	Recall MAE $\downarrow$	False-safe $\downarrow$	Regret@5 $\downarrow$
Tiny5M (Image)	Uniform	0.745 $\pm$ 0.024	0.031 $\pm$ 0.003	0.208 $\pm$ 0.038	0.117 $\pm$ 0.033
	EIGEN	0.823 $\pm$ 0.016	0.024 $\pm$ 0.002	0.129 $\pm$ 0.034	0.071 $\pm$ 0.021
GloVe (Text)	Uniform	0.666 $\pm$ 0.039	0.044 $\pm$ 0.005	0.282 $\pm$ 0.054	0.181 $\pm$ 0.054
	EIGEN	0.788 $\pm$ 0.026	0.031 $\pm$ 0.003	0.169 $\pm$ 0.041	0.096 $\pm$ 0.029
Msong (Audio)	Uniform	0.705 $\pm$ 0.035	0.038 $\pm$ 0.004	0.250 $\pm$ 0.044	0.151 $\pm$ 0.041
	EIGEN	0.799 $\pm$ 0.024	0.028 $\pm$ 0.003	0.157 $\pm$ 0.032	0.086 $\pm$ 0.024

6.5.1 Fidelity of Mini-DBs. We examine whether Mini-DBs preserve the configuration-performance landscape of the original database across three types of datasets. For each dataset, we construct an evaluation set of 70 configurations using stratified Latin hypercube sampling, with 10 configurations drawn for each supported index type. We compare our locality-aware sampling method with uniform random sampling. For each method, we independently construct three Mini-DBs of the same size and evaluate the same configurations on both the Mini-DBs and the original database.

Table 6 reports four metrics. First, we compute a constraint-aware version of Spearman rank correlation [31] between the configurations evaluated on a Mini-DB and those evaluated on the original database, where feasible configurations are ranked by QPS and infeasible configurations are placed after all feasible ones. Second, we report the mean absolute error in recall. Third, the false-safe rate is the fraction of Mini-DB-feasible configurations that violate the full-database recall constraint. Its denominator includes only configurations passing the Mini-DB recall threshold. Finally, we report transfer regret at five, defined as the relative QPS gap between the best feasible configuration among the five candidates selected by a Mini-DB and the best-known feasible configuration in the evaluation set.

Our sampling method consistently provides a more faithful approximation of the original tuning landscape. Across the evaluated datasets, locality-aware sampling achieves an average rank correlation of 0.803, compared with 0.705 for uniform sampling, while recall error decreases from 0.038 to 0.028. The false-safe rate decreases from 24.7% to 15.2%. The residual 15.2% is a candidate-level

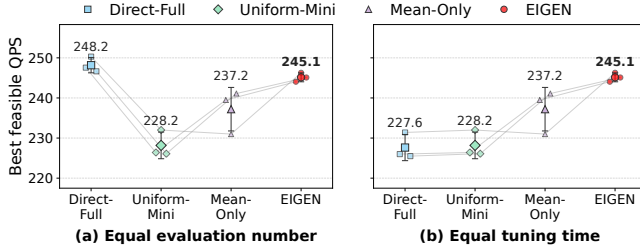


Figure 10: End-to-end ablation of Mini-DBs at recall 0.95. (a) equal evaluation number and (b) equal aggregate tuning time. Points show means and error bars show sample standard deviations over three runs.

transfer error before final validation, not the frequency of infeasible recommendations returned by the complete framework. EIGEN rejects candidates that fail the full-database recall check, even if they passed all Mini-DB checks. Its Regret5 is 0.084, compared with 0.150 for uniform sampling. These results demonstrate the fidelity of our Mini-DB construction method in preserving the tuning landscape, thereby supporting reliable configuration transfer.

6.5.2 End-to-End Contribution of Mini Databases. We next evaluate the end-to-end contribution of Mini-DBs by comparing four variants. DIRECT-FULL applies CALM directly to Tiny5M without constructing Mini-DBs. UNIFORM-MINI replaces our sampling method with uniform sampling while retaining all other components. MEAN-ONLY uses the Mini-DBs constructed by our method but ranks candidates only by their average normalized performance, removing the cross-database stability term. Finally, EIGEN denotes the complete framework. We consider two settings: (a) an equal number of physical evaluations and (b) an equal aggregate tuning-time budget set by the corresponding EIGEN run. For Mini-DB methods, search counts are summed across workers.

As shown in Figure 10, under the equal-evaluation-budget setting, EIGEN achieves 245.14 QPS, compared with 248.17 QPS for DIRECT-FULL. The difference is only 1.22%, showing that tuning on substantially smaller Mini-DBs preserves most of the configuration quality obtainable through direct full-scale tuning. The advantage becomes more pronounced under the equal tuning-time budget setting. Within the time required by EIGEN, DIRECT-FULL reaches only 227.63 QPS, while EIGEN achieves 245.14 QPS, corresponding to an improvement of 7.69%. Moreover, replacing locality-aware sampling with uniform sampling reduces the QPS from 245.14 to 228.17, a decrease of 6.92%. Removing the stability term lowers mean feasible QPS from 245.14 to 237.20 and increases the across-run sample standard deviation from 1.11 to 5.43 QPS. Equivalently, the complete framework reduces the standard deviation by 79.6%, with a coefficient of variation of only 0.45% across the three runs. Thus, stability-aware aggregation improves both configuration quality and run-to-run consistency in this experiment, supporting repeatable recommendations across the tested runs.

6.5.3 Design Choices of CALM. We investigate whether the effectiveness of CALM stems merely from the use of an LLM or from its structured optimization workflow. We compare the complete CALM with three variants. GLOBAL-LLM removes the region partitioning mechanism and generates configurations over the entire

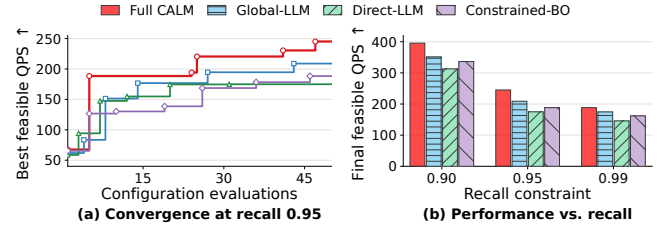


Figure 11: Ablation of the design choices in CALM. (a) Convergence curves under a recall constraint of 0.95. The horizontal axis counts physical configuration evaluations summed across all Mini-DBs. (b) Feasible QPS across different recall constraints.

heterogeneous search space, while retaining the same iterative feedback loop as CALM. DIRECT-LLM further removes the structured optimization workflow and directly prompts the same LLM at each iteration to recommend the next candidate configurations based only on the raw evaluation history. CONSTRAINED-BO replaces the LLM-based optimizer with a conventional constrained Bayesian optimizer using the same search space. We conduct the experiment on Tiny5M under recall constraints of {0.90, 0.95, 0.99}.

Figure 11(a) presents the convergence curves under $\rho = 0.95$. Full CALM maintains a higher best-so-far feasible QPS throughout most of the tuning process and ultimately achieves the highest QPS among all methods. In comparison, GLOBAL-LLM improves progressively through iterative feedback but remains below CALM throughout most of the tuning process. This gap demonstrates the benefit of partitioning the heterogeneous, index-specific configuration space into more coherent regions, which allows the optimizer to concentrate its search on promising subspaces. DIRECT-LLM achieves a relatively strong result within the first few evaluations but quickly reaches a performance plateau. This behavior suggests that semantic priors can identify reasonable configurations early, while raw evaluation history alone provides less effective guidance than CALM’s region selection and surrogate screening under the same budget. CONSTRAINED-BO exhibits slower improvement and achieves only moderate final performance under the same evaluation budget. Although it can gradually learn the relationship between configurations, recall, and QPS, it requires more observations to model the mixed and conditional search space effectively. Figure 11(b) reports the feasible QPS under different recall constraints. CALM consistently achieves the best result across all three thresholds. These results show that CALM’s effectiveness does not arise solely from introducing an LLM into the tuning process. These variants isolate design choices within our optimizer.

6.6 Sensitivity and Robustness Analysis

We next study whether the effectiveness of EIGEN is sensitive to its main design parameters or the choice of LLM backbone.

6.6.1 Number and Size of Mini-DBs. We first vary the number of Mini-DBs over 1, 2, 3, 4, 5 while fixing the size of each Mini-DB to 10% of Tiny5M. We then vary the sampling ratio over 1%, 5%, 10%, 20%, while fixing the number of Mini-DBs to three. The recall constraint is fixed at 0.95 in both experiments.

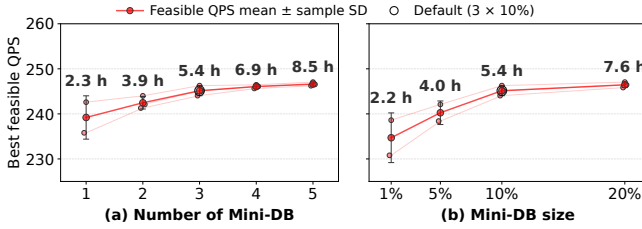


Figure 12: Sensitivity Analysis of Mini-DBs.

Figure 12 shows the results. Increasing the number of Mini-DBs from one to three improves the final QPS from 239.20 to 245.14 and reduces the standard deviation across feasible runs from 4.81 to 1.11. This improvement shows that multiple independent Mini-DBs reduce the risk of selecting configurations that perform well only on a particular sample. Increasing the number further to five yields only an additional 0.60% improvement, while increasing tuning time by 56.85%. Thus, three Mini-DBs provide a favorable balance between transfer reliability and tuning cost. A similar saturation trend is observed for Mini-DB size. Increasing the sampling ratio from 1% to 10% improves QPS by 4.45%. Increasing the ratio to 20%, however, provides only a further 0.53% improvement at an additional tuning-time cost of 40.56%. These results support the default setting of three 10% Mini-DBs, which lies near the knee of the quality–cost curve.

Table 7: Configuration quality, total LLM time, and total API cost per EIGEN tuning run on GloVe at recall threshold 0.95.

Model	Reasoning Effort	Best QPS	LLM Time (s)	Token Cost (USD)
DeepSeek-V4-Pro-0813	high	322.11	18,882.51	6.71
gpt-5.4-2026-03-05	high	342.68	20,753.60	24.27
gpt-5.4-2026-03-05	medium	315.84	6,840.83	11.66
gpt-5.4-2026-03-05	low	305.29	4,230.76	8.56
claude-opus-4-7	adaptive	356.33	15,311.99	43.74
gpt-5.6-sol	high	359.34	14,110.81	27.96

6.6.2 Robustness to LLM Backbones. Since EIGEN employs LLMs to propose new configurations, we investigate whether its effectiveness depends on a particular LLM backbone or reasoning effort. Each backbone is used for both candidate generation and surrogate prediction. We evaluate four representative models: GPT-5.4 [24], DeepSeek-V4-pro [8], Claude-opus-4.7 [1], and GPT-5.6-sol [25]. For GPT-5.4, we further vary the reasoning effort among *low*, *medium*, and *high*. We use the same GloVe dataset and a recall constraint of 0.95 for all backbones, while keeping all model-independent settings identical.

As shown in Table 7, EIGEN consistently identifies high-quality feasible configurations with all backbones. This consistency indicates that the effectiveness of EIGEN is not tied to a particular model or provider. We also observe a clear trade-off among configuration quality, inference latency, and monetary cost. More capable backbones and higher reasoning effort generally yield better candidate configurations and higher final feasible QPS, but also require longer inference time and incur greater API cost. The GPT-5.4 medium setting used in the main experiments offers a practical balance among configuration quality, inference latency, and API cost.

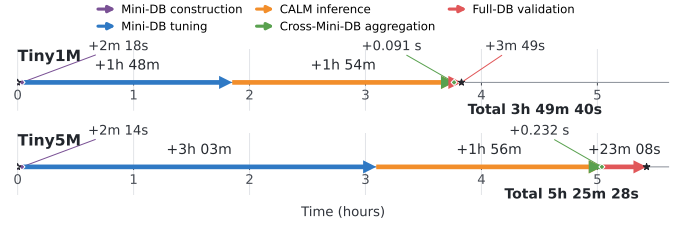


Figure 13: End-to-end cost breakdown.

6.7 Overhead Analysis

Figure 13 breaks down the aggregate tuning time on Tiny1M and Tiny5M into Mini-DB construction, Mini-DB tuning, CALM inference, cross-Mini-DB aggregation, and full-database validation. Mini-DB tuning includes local configuration evaluations and physical cross-Mini-DB evaluations; aggregation covers numerical scoring and candidate ranking. Following Section 6.1, worker times are summed. Full-database validation includes data loading, required index construction, and query execution for the transferred shortlist, with database-state reuse disabled.

Mini-DB tuning and CALM inference dominate the runtime. From Tiny1M to Tiny5M, Mini-DB tuning time increases by 1.69 $\times$, whereas inference remains nearly constant at approximately 1 h 55 min. Consequently, the LLM share of total tuning time decreases from about 49.6% to 35.6%. Under a fixed search budget, the LLM processes task metadata and bounded configuration–performance histories whose sizes do not scale directly with database cardinality, whereas physical evaluation costs grow with data volume. The relative LLM overhead is therefore amortized as the database grows, making semantic guidance more economical when each physical evaluation is more expensive. Mini-DB construction takes approximately two minutes on both datasets, and numerical aggregation takes less than one second. Full-database validation increases from 3 min 49 s to 23 min 8 s. Overall, total tuning time grows from 3 h 49 min 40 s to 5 h 25 min 28 s (1.42 $\times$) over a fivefold increase in database size.

7 CONCLUSION

We presented EIGEN, an end-to-end framework for vector database tuning. It combines representative Mini-DBs with CALM, a constraint-aware LLM-guided optimizer, to reduce both configuration evaluation cost and search effort while jointly tuning system, index, and search parameters. By decoupling database-specific configuration descriptions and execution interfaces from the core tuning procedure, EIGEN supports adaptation across database engines, workloads, and hardware environments without task-specific offline training. Experiments across six datasets show that EIGEN achieves comparable or higher feasible QPS with an average tuning-time speedup of 2.25 $\times$ over the fastest competing method, with further evaluations demonstrating its effectiveness across different hardware settings and database engines.

Use of AI Tools

The authors used ChatGPT and Codex to assist with manuscript editing, formatting, and coding. The authors take full responsibility for the accuracy, integrity, and originality of this work.

REFERENCES

- [1] Anthropic. 2026. Introducing Claude Opus 4.7. Retrieved September 7, 2026 from <https://www.anthropic.com/news/claude-opus-4-7>
- [2] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Information Systems* 87 (2020), 101374. <https://doi.org/10.1016/j.is.2019.02.006>
- [3] Baoqing Cai, Yu Liu, Ce Zhang, Guangyu Zhang, Ke Zhou, Li Liu, Chunhua Li, Bin Cheng, Jie Yang, and Jiashu Xing. 2022. HUNTER: An Online Cloud Database Hybrid Tuning System for Personalized Requirements. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) (SIGMOD '22). Association for Computing Machinery, New York, NY, USA, 646–659. <https://doi.org/10.1145/3514221.3517882>
- [4] Moses S. Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *Proceedings of the Thirty-Fourth Annual ACM Symposium on Theory of Computing (STOC '02)*. Association for Computing Machinery, New York, NY, USA, 380–388. <https://doi.org/10.1145/509907.509965>
- [5] Geon Choi, Hoeun Lee, and Jaeyoung Do. 2026. Exploiting Structural Properties for Efficient Constraint-Aware HNSW Hyperparameter Tuning. arXiv:2607.04630 [cs.DB] <https://arxiv.org/abs/2607.04630>
- [6] William G. Cochran. 1977. *Sampling Techniques* (3rd ed.). John Wiley & Sons, New York.
- [7] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the Twentieth Annual Symposium on Computational Geometry*. Association for Computing Machinery, 253–262. <https://doi.org/10.1145/997817.997857>
- [8] DeepSeek. 2026. DeepSeek API: Models & Pricing. Retrieved September 7, 2026 from https://api-docs.deepseek.com/quick_start/pricing
- [9] Hao Duan, Yitong Song, Bin Yao, and Anqi Liang. 2025. PGTuner: An Efficient Framework for Automatic and Transferable Configuration Tuning of Proximity Graphs. *Proc. ACM Manag. Data* 3, 4, Article 261 (Sept. 2025), 27 pages. <https://doi.org/10.1145/3749179>
- [10] Victor Giannakouris and Immanuel Trummer. 2025. λ -Tune: Harnessing Large Language Models for Automated Database System Tuning. *Proceedings of the ACM on Management of Data* 3, 1, Article 2 (2025), 26 pages. <https://doi.org/10.1145/3709652>
- [11] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML '20)*. JMLR.org, Article 364, 10 pages.
- [12] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. <https://doi.org/10.1109/TBDDATA.2019.2921572>
- [13] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. <https://doi.org/10.1109/TPAMI.2010.57>
- [14] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in one billion vectors: Re-rank with source coding. *Acoustics, Speech, and Signal Processing, 1988. ICASSP-88, 1988 International Conference on* (02 2011). <https://doi.org/10.1109/ICASSP.2011.5946540>
- [15] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2024. GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1939–1952. <https://doi.org/10.14778/3659437.3659449>
- [16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [17] Guoliang Li, Xuanhe Zhou, Shifu Li, and Bo Gao. 2019. QTune: a query-aware database tuning system with deep reinforcement learning. *Proc. VLDB Endow.* 12, 12 (Aug. 2019), 2118–2130. <https://doi.org/10.14778/3352063.3352129>
- [18] Tennyson Liu, Nicolás Astorga, Nabeel Seedat, and Mihaela van der Schaar. 2024. Large Language Models to Enhance Bayesian Optimization. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024*. OpenReview.net. <https://openreview.net/forum?id=OOxotBmGol>
- [19] Wei-Liem Loh. 1996. On Latin hypercube sampling. *Annals of Statistics* 24 (1996), 2058–2080. <https://api.semanticscholar.org/CorpusID:121050124>
- [20] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>
- [21] Marius Muja and David G. Lowe. 2009. Fast Approximate Nearest Neighbors with Automatic Algorithm Configuration. In *Proceedings of the Fourth International Conference on Computer Vision Theory and Applications*, Vol. 1. SciTePress, 331–340. <https://doi.org/10.5220/0001787803310340>
- [22] NVIDIA. [n.d.]. Tuning Indexes. NVIDIA cuVS Documentation. <https://docs.nvidia.com/cuvs/getting-started/introduction/tuning-indexes> Section: Globally partitioned indexes. Accessed September 24, 2026.
- [23] Gabriele Onorato. 2024. Bayesian Optimization for Hyperparameters Tuning in Neural Networks. arXiv:2410.21886 [cs.LG] <https://arxiv.org/abs/2410.21886>
- [24] OpenAI. 2026. GPT-5.4 Model. API documentation. Retrieved September 7, 2026 from <https://developers.openai.com/api/docs/models/gpt-5.4>
- [25] OpenAI. 2026. Previewing GPT-5.6 Sol: A Next-Generation Model. Retrieved September 7, 2026 from <https://openai.com/index/previewing-gpt-5-6-sol/>
- [26] pgvector contributors. [n.d.]. pgvector: Open-Source Vector Similarity Search for Postgres. Software and documentation, version 0.6.2. Retrieved September 7, 2026 from <https://github.com/pgvector/pgvector/tree/v0.6.2>
- [27] PostgreSQL Global Development Group. [n.d.]. PostgreSQL Documentation: Server Configuration. Retrieved September 7, 2026 from <https://www.postgresql.org/docs/current/runtime-config.html>
- [28] Qdrant. [n.d.]. Qdrant: Vector Similarity Search Engine. Software repository, version 1.16.0. Retrieved September 7, 2026 from <https://github.com/qdrant/qdrant/tree/v1.16.0>
- [29] Qdrant. [n.d.]. Vector-db-benchmark: Framework for benchmarking vector search engines. Software repository. Retrieved September 7, 2026 from <https://github.com/qdrant/vector-db-benchmark>
- [30] Andrej Schwanke, Lyubomir Ivanov, David Salinas, Frank Hutter, and Arber Zela. 2026. Multi-Objective Hierarchical Optimization with Large Language Models. arXiv:2601.13892 [cs.LG] <https://arxiv.org/abs/2601.13892>
- [31] C. Spearman. 2010. The proof and measurement of association between two things. *International Journal of Epidemiology* 39, 5 (10 2010), 1137–1150. <https://doi.org/10.1093/ije/dyq191> arXiv:https://academic.oup.com/ije/article-pdf/39/5/1137/18481215/dyq191.pdf
- [32] Ján Suchal and Pavol Návrát. 2010. Full Text Search Engine as Scalable k-Nearest Neighbor Recommendation System. In *Artificial Intelligence in Theory and Practice III*, Max Bramer (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 165–173.
- [33] Immanuel Trummer. 2022. DB-BERT: A Database Tuning Tool that “Reads the Manual”. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) (SIGMOD '22). Association for Computing Machinery, New York, NY, USA, 190–203. <https://doi.org/10.1145/3514221.3517843>
- [34] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *Proceedings of the 2017 ACM International Conference on Management of Data (Chicago, Illinois, USA) (SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 1009–1024. <https://doi.org/10.1145/3035918.3064029>
- [35] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (Virtual Event, China) (SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2614–2627. <https://doi.org/10.1145/3448016.3457550>
- [36] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proc. VLDB Endow.* 14, 11 (July 2021), 1964–1978. <https://doi.org/10.14778/3476249.3476255>
- [37] Tiannuo Yang, Wen Hu, Wangqi Peng, Yusen Li, Jianguo Li, Gang Wang, and Xiaoguang Liu. 2024. VDTuner: Automated Performance Tuning for Vector Data Management Systems. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 4357–4369. <https://doi.org/10.1109/ICDE60146.2024.00332>
- [38] Ji Zhang, Yu Liu, Ke Zhou, Guoliang Li, Zhili Xiao, Bin Cheng, Jiashu Xing, Yangtao Wang, Tianheng Cheng, Li Liu, Minwei Ran, and Zekang Li. 2019. An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning. In *Proceedings of the 2019 International Conference on Management of Data (Amsterdam, Netherlands) (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 415–432. <https://doi.org/10.1145/3299869.3300085>
- [39] Xinyi Zhang, Hong Wu, Zhuo Chang, Shuwei Jin, Jian Tan, Feifei Li, Tieying Zhang, and Bin Cui. 2021. ResTune: Resource Oriented Tuning Boosted by Meta-Learning for Cloud Databases. In *Proceedings of the 2021 International Conference on Management of Data (Virtual Event, China) (SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2102–2114. <https://doi.org/10.1145/3448016.3457291>
- [40] Yuqing Zhu, Jianxun Liu, Mengying Guo, Yungang Bao, Wenlong Ma, Zhuoyue Liu, Kunpeng Song, and Yingchun Yang. 2017. BestConfig: tapping the performance potential of systems via automatic configuration tuning. In *Proceedings of the 2017 Symposium on Cloud Computing (Santa Clara, California) (SoCC '17)*. Association for Computing Machinery, New York, NY, USA, 338–350. <https://doi.org/10.1145/3127479.3128605>